# Multivariable Control Problems

**Coursework**

Matteo Gennari, GROUP 4

February 20, 2026

# Contents

**1 Control System Analysis** **3**
1.1 System zeros . . . 3
1.2 System perturbation . . . 7

**2 MIMO-Systems** **10**
2.1 Proof of equivalence . . . 10
2.2 System zero and down squaring . . . 11
2.3 Stabilization via Youla-controller . . . 12

**3 Stability and LFT** **14**
3.1 Stability and LFT . . . 14
3.2 Sensitivity integrals . . . 22

**4 Coprime factorization and Youla Parametrization** **25**
4.1 HIMAT example system analysis . . . 25
4.2 Coprime factorization . . . 27
4.3 Youla parametrization . . . 27

# Task 1: Control System Analysis

### 1.1: System zeros

This chapter will demonstrate a procedure to find the zeros and the corresponding initial conditions of a MIMO system in state-space representation. Two cases will be discussed: a system with the same number of inputs and outputs (square system) and a system with a different number of inputs and outputs.

It's possible to re-write the state equations of a system using the Rosenbrock matrix:

$$\boldsymbol{P}(s)\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{u}\end{bmatrix}=\begin{bmatrix}\boldsymbol{0}\\ \boldsymbol{y}\end{bmatrix},\quad \boldsymbol{P}(s)=\begin{bmatrix}s\boldsymbol{I}-\boldsymbol{A} & -\boldsymbol{B}\\ \boldsymbol{C} & \boldsymbol{D}\end{bmatrix}. \tag{1.1}$$

The zeros of a system are the values $s = z$ such that $P(z)$ drops rank. If the system is square, the Rosenbrock matrix is square as well, and therefore the zeros are found as the solution of the generalized eigenvalue problem:

$$(z\boldsymbol{I}_g-\boldsymbol{M})\begin{bmatrix}\boldsymbol{x}_z\\ \boldsymbol{u}_z\end{bmatrix}=\boldsymbol{0}. \tag{1.2}$$

$$\boldsymbol{M}=\begin{bmatrix}\boldsymbol{A} & \boldsymbol{B}\\ \boldsymbol{C} & \boldsymbol{D}\end{bmatrix};\quad \boldsymbol{I}_g=\begin{bmatrix}\boldsymbol{I} & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{0}\end{bmatrix}. \tag{1.3}$$

This type of problem can be easily solved in MATLAB by using the function `[V, D] = eig(M, I_g)`. The matrix `D` has the eigenvalues $z$ on the diagonal, while the matrix `V` has as columns the input vectors associated to each zero. From these vectors, the state direction and input direction can be obtained as shown in Eq. (1.4).

$$\begin{bmatrix}\boldsymbol{x}_z\\ \boldsymbol{u}_z\end{bmatrix}. \tag{1.4}$$

The transfer function provided for testing is:

$$\boldsymbol{G}(s)=\begin{bmatrix}\frac{1}{s} & \frac{1}{s(s+1)}\\ \frac{1}{s+1} & \frac{1}{s+1}\end{bmatrix} \tag{1.5}$$

By using the described procedure, two zeros were identified as reported in Table 1.1.

This way, all the invariant zeros of the system are found: for a non-minimal realization of the system the input/output decoupling zeros will also be identified.

In case it's only necessary to find the transmission zeros, the same procedure can be applied after reducing the system to a minimal realization. The procedure discussed in [3] was used to find such minimal realization. The minimal realization of $G(s)$ is:

Table 1.1: System Invariant Zeros and Associated Directions

| **Variable** | **Mode 1** | **Mode 2** |
|---|---|---|
| Invariant Zero ($z$) | $-0.2719 \cdot 10^{-15}$ | $0$ |
| State Direction ($\boldsymbol{x}_0$) | $\begin{pmatrix} -1 \\ -0 \\ 1 \\ 0 \end{pmatrix}$ | $\begin{pmatrix} 1 \\ 0 \\ -1 \\ 0 \end{pmatrix}$ |
| Input Direction ($\boldsymbol{u}_0$) | $\begin{pmatrix} 0.2719 \\ -0.2719 \end{pmatrix} \cdot 10^{-15}$ | $\begin{pmatrix} -0.5206 \\ 0.5206 \end{pmatrix} \cdot 10^{-15}$ |

$$\bar{\boldsymbol{A}} = \begin{bmatrix} -0.4725 & -0.0005 & -0.0274 \\ 0.0328 & -0.6657 & 0.3878 \\ 0.0751 & 0.0169 & -0.0054 \end{bmatrix}, \quad \bar{\boldsymbol{B}} = \begin{bmatrix} 0.8838 & 1.1165 \\ 0.6693 & -0.6348 \\ 0.6172 & 0.6126 \end{bmatrix} \tag{1.6}$$

$$\bar{\boldsymbol{C}} = \begin{bmatrix} -0.0507 & 0.5008 & 0.8641 \\ 0.4826 & 0.1234 & -0.0432 \end{bmatrix}, \quad \bar{\boldsymbol{D}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{1.7}$$

The zeros obtained by using the same procedure with this new system are shown in Table 1.2. The complete procedure for square systems can be found in the code named `Ex_2_1_1.m`.

Table 1.2: System Transmission Zeros and Associated Directions

| **Variable** | **Mode 1** |
|---|---|
| Transmission Zero ($z$) | $4.8409 \cdot 10^{-16}$ |
| State Direction ($\boldsymbol{x}_0$) | $\begin{pmatrix} -0.3092 \\ 1 \\ -0.5978 \end{pmatrix}$ |
| Input Direction ($\boldsymbol{u}_0$) | $\begin{pmatrix} 0.6959 \\ -0.6959 \end{pmatrix}$ |

For a non-square system, the generalized eigenvalue problem cannot be solved as easily, since the matrix $\boldsymbol{P}(s)$ is not square. In order to find the values of $z$, we can check the

singular values of the Rosenbrock matrix and see where the smallest becomes zero. The problem is that it's impossible to check the entirety of the complex plane: it is necessary to define a grid of points to check and for each point compute the Rosenbrock matrix and perform an SVD, which is very computationally expensive. Moreover, it's necessary to define a threshold below which a point is accepted as a zero. The system provided to test the MATLAB procedure was:

$$\boldsymbol{G}(s) = \frac{1}{(s+1)(s+2)(s-1)} \begin{bmatrix} (s-1)(s+2) & 0 & (s-1)^2 \\ -(s+1)(s+2) & (s-1)(s+1) & (s-1)(s+1) \end{bmatrix} \tag{1.8}$$

The results obtained with this procedure are reported in Table 1.3.

Table 1.3: System Invariant Zeros and Associated Directions

| **Variable** | **Mode 1** | **Mode 2** |
|---|---|---|
| Invariant Zero ($z$) | $-1$ | $1$ |
| State Direction ($\boldsymbol{x}_0$) | $\begin{pmatrix} 0.1562 & 0.3498 \\ -0.1562 & -0.3498 \\ 0 & 0 \\ -0.1472 & 0.6717 \\ 0.4595 & 0.0279 \\ 0 & 0 \end{pmatrix}$ | $\begin{pmatrix} -0.2980 & 0.2181 \\ 0.2980 & -0.2181 \\ 0 & 0 \\ 0.4398 & 0.1840 \\ 0.4398 & 0.1840 \\ 0 & 0 \end{pmatrix}$ |
| Input Direction ($\boldsymbol{u}_0$) | $\begin{pmatrix} 0.2083 & 0.4664 \\ -0.8150 & 0.1775 \\ 0.1041 & 0.2332 \end{pmatrix}$ | $\begin{pmatrix} 0 & 0 \\ 0.2836 & -0.8043 \\ 0.5959 & 0.4362 \end{pmatrix}$ |

This procedure, contained in the code named `Ex_2_1_2_opt.m`, is not robust: the results depend on the grid chosen and on the threshold. These parameters are very system-dependent and cannot be chosen accurately beforehand.

Another procedure is possible which allows to find only the transmission zeros of the system. It is based on the zero polynomial of the system. From [5]:

> The zero polynomial $z(s)$, corresponding to a minimal realization of the system, is the greatest common divisor of all the numerators of all order-$r$ minors of $\boldsymbol{G}(s)$, where $r$ is the normal rank of $\boldsymbol{G}(s)$, provided that these minors have been adjusted in such a way as to have the pole polynomial $\phi(s)$ as their denominator.

The procedure implemented in MATLAB uses the *Symbolic Math Toolbox* to compute the pole polynomial and the zero polynomial.

The system's pole polynomial $\phi(s)$ was computed as the least common multiple of the denominators of all minors of $\boldsymbol{G}(s)$. Then, the system's zero polynomial $z(s)$ was computed as the greatest common divisor of the numerators of all maximal-order minors of $\boldsymbol{G}(s)$, once adjusted by the system's pole polynomial $\phi(s)$. The roots of $z(s)$ correspond to the transmission zeros.

The associated initial state ($\boldsymbol{x}_0$) and input ($\boldsymbol{u}_0$) directions are extracted numerically. For each identified zero $z$, the directions are found by computing the null space of the Rosenbrock system matrix using Singular Value Decomposition. The right singular vectors corresponding to null singular values are used to define a basis for the zero direction. The results obtained through this procedure are displayed in Table 1.4.

Table 1.4: System Transmission Zeros and Associated Directions

| **Variable** | **Mode 1** |
|---|---|
| Invariant Zero ($z$) | 1 |
| State Direction ($\boldsymbol{x}_0$) | $\begin{pmatrix} 0.2980 & -0.2181 \\ -0.2980 & 0.2181 \\ 0 & 0 \\ -0.4398 & -0.1840 \\ -0.4398 & -0.1840 \\ 0 & 0 \end{pmatrix}$ |
| Input Direction ($\boldsymbol{u}_0$) | $\begin{pmatrix} 0 & 0 \\ -0.2836 & -0.8043 \\ -0.5959 & 0.4362 \end{pmatrix}$ |

It's interesting to observe that the basis for the initial state and input directions are two dimensional. This is because the Rosenbrock matrix has more columns than rows (3 inputs and 2 outputs), therefore its column null space has dimension 1 at all frequencies except at zero, where it has dimension 2.

The reader may also notice how in this procedure it's not required to compute the minimal state-space realization of the system, since it works directly with its transfer function. Since it was required, the minimal realization (obtained with the same procedure as before) is shown in Eq. 1.9 and Eq. 1.10. This procedure for non-square systems can be seen in the code called `Ex_2_1_2_opt.m`.

$$\bar{\boldsymbol{A}} = \begin{bmatrix} -0.0893 & -0.0001 & -0.0410 & 0.0096 \\ 0.0001 & -0.1231 & -0.0180 & -0.0419 \\ 0.0018 & 0.0014 & -0.4781 & -0.0066 \\ 0.0001 & 0.0012 & -0.0089 & 0.6038 \end{bmatrix}, \quad \bar{\boldsymbol{B}} = \begin{bmatrix} -0.0798 & -0.5097 & -5.0649 \\ 0.0010 & 2.1618 & 0.1157 \\ 0.8563 & -0.0301 & -1.4875 \\ -1.4302 & -0.0146 & -0.0194 \end{bmatrix} \tag{1.9}$$

$$\bar{\boldsymbol{C}} = \begin{bmatrix} -0.0237 & 0.0067 & 0.5425 & 0.0041 \\ -0.0084 & 0.0260 & -0.0032 & 0.4261 \end{bmatrix}, \quad \bar{\boldsymbol{D}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{1.10}$$

### 1.2: System perturbation

Given the following system:

$$\boldsymbol{G}(s) = \begin{bmatrix} 7 & 8 \\ 6 & 7 \end{bmatrix} \begin{bmatrix} \frac{1}{s+1} & 0 \\ 0 & \frac{2}{s+2} \end{bmatrix} \begin{bmatrix} 7 & 8 \\ 6 & 7 \end{bmatrix}^{-1}, \tag{1.11}$$

With controller $\boldsymbol{K} = -\boldsymbol{I}$ in positive feedback configuration as seen in Figure 1.1. In order to simplify the calculations, it was considered an equivalent system with controller $\boldsymbol{K} = \boldsymbol{I}$ in negative feedback configuration. It was required to study the internal stability of this system. From [4]:

> A given linear-time invariant feedback control system is internally stable, if all transfer functions from any input (including disturbance inputs) to any output is stable (i.e. poles of all transfer functions lie in the open LHP).

In particular, for a well posed system we have to consider the matrices:

$$\begin{bmatrix} \boldsymbol{I} & \boldsymbol{K}(s) \\ -\boldsymbol{G}(s) & \boldsymbol{I} \end{bmatrix}^{-1} = \begin{bmatrix} (\boldsymbol{I} + \boldsymbol{K}\boldsymbol{G}(s))^{-1} & -\boldsymbol{K}(s)(\boldsymbol{I} + \boldsymbol{G}\boldsymbol{K}(s))^{-1} \\ \boldsymbol{G}(s)(\boldsymbol{I} + \boldsymbol{K}\boldsymbol{G}(s))^{-1} & (\boldsymbol{I} + \boldsymbol{G}\boldsymbol{K}(s))^{-1} \end{bmatrix} \in \mathcal{RH}_\infty \tag{1.12}$$

For each of the transfer functions, it's necessary to check their rationality, stability and properness. To check for stability, a custom function `find_poles` was written, which takes as input a transfer function matrix and returns its poles. It uses the same procedure explained in chapter 1.1 to compute the pole polynomial and computes its roots to find the system poles. All the poles found for all four matrices are inside the left-hand plane, meaning that they are all stable. To check for properness, the MATLAB built-in function `isProper` was used, which confirmed the properness of all four matrices. Finally, to check for rationality, it was used the function `hasdelay`, which confirmed that none of the functions had delays.

In this special case, however, checking all of these conditions was not necessary. This is because our controller $\boldsymbol{K} \in \mathcal{RH}_\infty$. Therefore, the 1-DOF control configuration is internally stable if it's well posed and if $(\boldsymbol{I} + \boldsymbol{G}\boldsymbol{K}(s))^{-1}\boldsymbol{G}(s) \in \mathcal{RH}_\infty$.

Consider now the plant perturbed by additive uncertainty. The perturbed plant equation is:

$$\boldsymbol{G}_p = \boldsymbol{G} + \boldsymbol{W}_1 \boldsymbol{\Delta} \boldsymbol{W}_2 \tag{1.13}$$

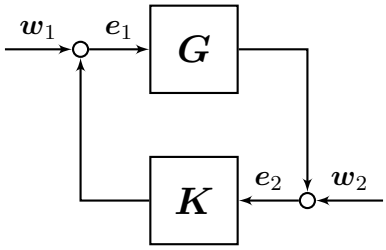


Figure 1.1: Feedback control loop.

To ensure robust stability of this system, it's necessary to compute $\gamma$ such that:

$$\|\boldsymbol{W}_2\boldsymbol{K}\boldsymbol{S}_0\boldsymbol{W}_1\|_\infty = \|\boldsymbol{W}_2\boldsymbol{K}(\boldsymbol{I}+\boldsymbol{G}\boldsymbol{K}(s))^{-1}\boldsymbol{W}_1\|_\infty \leq \gamma \tag{1.14}$$

The perturbation $\boldsymbol{\Delta}$ has to satisfy $\|\boldsymbol{\Delta}\| < \frac{1}{\gamma}$, which therefore represents a stability margin for the system. In order to find this margin and the frequency in which the worst-case perturbation occurs, we can use the MATLAB function `hinfnorm`. Using this procedure, the margin found was $\frac{1}{\gamma} = 0.0611$ at frequency $\omega = 2.8327\ \frac{\text{rad}}{\text{s}}$.

Then, we can use the function `sigma` to plot the Singular Values as function of frequency and overlay the worst-case found. The resulting plot is shown in Figure 1.2.

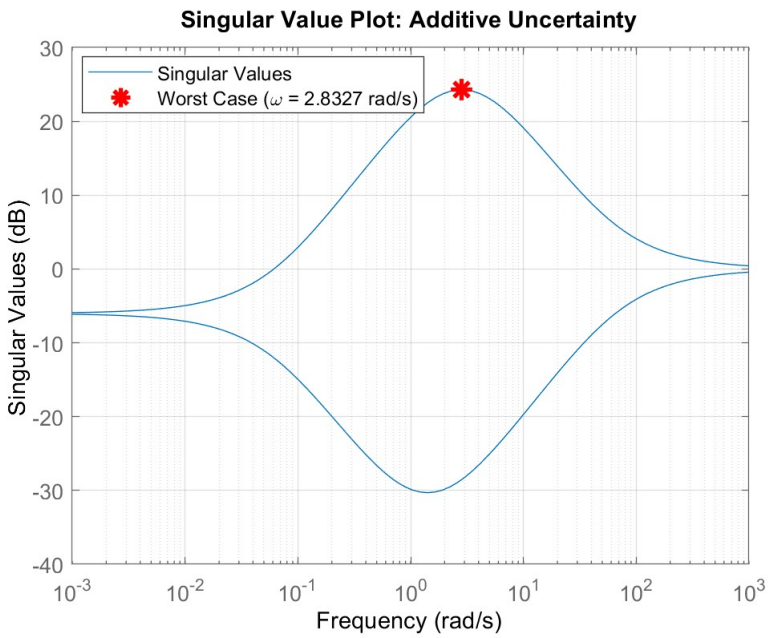


Figure 1.2: Singular Value plot for the system with additive uncertainty.

Instead, if we have multiplicative output uncertainty the process is quite similar. The perturbed plant equation is:

$$\boldsymbol{G}_p = (\boldsymbol{I} + \boldsymbol{W}_1 \boldsymbol{\Delta} \boldsymbol{W}_2)\boldsymbol{G} \tag{1.15}$$

In this case, for robust stability we have to compute $\gamma$:

$$\|\boldsymbol{W}_2 \boldsymbol{T}_0 \boldsymbol{W}_1\|_\infty = \|\boldsymbol{W}_2 \boldsymbol{G}\boldsymbol{K}(\boldsymbol{I} + \boldsymbol{G}\boldsymbol{K}(s))^{-1}\boldsymbol{W}_1\|_\infty \leq \gamma \tag{1.16}$$

And again, the perturbation $\boldsymbol{\Delta}$ has to satisfy $\|\boldsymbol{\Delta}\| < \frac{1}{\gamma}$. In this case, the margin found was $\frac{1}{\gamma} = 0.0612$ at frequency $\omega = 2.8270\,\frac{\text{rad}}{\text{s}}$, which are very close to the previous case. The resulting Singular Value plot is shown in Figure 1.3.

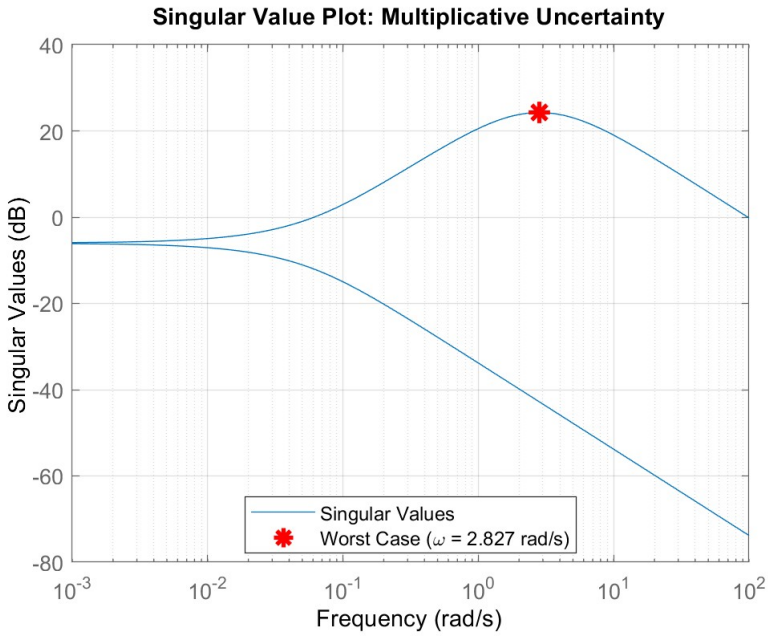


Figure 1.3: Singular Value plot for the system with multiplicative uncertainty.

The procedure explained in chapter 8.6.1 of [5] shows how to find the smallest destabilizing perturbation of a system, which satisfies $\|\boldsymbol{\Delta}'\|_\infty = 1/\gamma$. The formula is:

$$\boldsymbol{\Delta}' = -\frac{1}{\gamma}\boldsymbol{V}\boldsymbol{U}^H \tag{1.17}$$

This returns a matrix such that $\bar{\sigma}(\Delta'(j\omega^*)) = 1/\gamma$. Since it was required that the greatest singular value was equal to gamma, the term $1/\gamma$ in Eq. 1.17 was substituted with $\gamma$. This was done for both the additive and multiplicative uncertainty, yielding a system which is not stable, and therefore not in $\mathcal{RH}_\infty$. This was proved by confirming that the matrices of 1.12 had poles in the open right-half plane by using the two perturbed plants in the place of $\boldsymbol{G}(s)$. The procedure and results discussed in this section can be found in code `Ex_2_1_3.m`.

# Task 2: MIMO-Systems

### 2.1: Proof of equivalence

Given a system in general control configuration with generalized plant $\boldsymbol{P}$ and controller $\boldsymbol{K}$, suppose the exogenous input $\boldsymbol{w}$ is white noise with unit intensity. That is:

$$E\left\{\boldsymbol{w}(t)\boldsymbol{w}(\tau)^T\right\} = \boldsymbol{I}\delta(t-\tau) \tag{2.1}$$

The error signal is given by:

$$\boldsymbol{z} = \mathcal{F}_l(\boldsymbol{P},\boldsymbol{K})\boldsymbol{w} = \boldsymbol{F}\boldsymbol{w} \tag{2.2}$$

The expected power in the error signal $\boldsymbol{z}$ is then given by:

$$E\left\{\lim_{T\to\infty}\frac{1}{2T}\int_{-T}^{T}\boldsymbol{z}(t)^T\boldsymbol{z}(t)\,dt\right\} = E\left\{\lim_{T\to\infty}\frac{1}{2T}\int_{-T}^{T}\operatorname{trace}\left[\boldsymbol{z}(t)^T\boldsymbol{z}(t)\right]dt\right\} \tag{2.3}$$

Which are equivalent because $\boldsymbol{z}(t)^T\boldsymbol{z}(t)$ is a scalar. The following step was not proved but it's assumed that, under certain conditions, it's possible to move the expected value inside both the limit and the integral, revealing the autocorrelation function $R_{zz}$:

$$\lim_{T\to\infty}\frac{1}{2T}\int_{-T}^{T}\operatorname{trace}\left\{E\left[\boldsymbol{z}(t)^T\boldsymbol{z}(t)\right]\right\}dt = \lim_{T\to\infty}\frac{1}{2T}\int_{-T}^{T}\operatorname{trace}\left[R_{zz}(t,t)\right]dt \tag{2.4}$$

The auto-correlation at time instant $t$ can be written by using the convolutional integral:

$$\begin{aligned} R_{zz}(t,t) &= E\left\{\left(\int_{-\infty}^{\infty}\mathbf{H}(\tau_1)\boldsymbol{\omega}(t-\tau_1)\,d\tau_1\right)\cdot\left(\int_{-\infty}^{\infty}\mathbf{H}(\tau_2)\boldsymbol{\omega}(t-\tau_2)\,d\tau_2\right)^T\right\} \\ &= \int_{-\infty}^{\infty}\int_{-\infty}^{\infty}\mathbf{H}(\tau_1)E\left[\boldsymbol{\omega}(t-\tau_1)\boldsymbol{\omega}^T(t-\tau_2)\right]\mathbf{H}^T(\tau_2)\,d\tau_1\,d\tau_2 \end{aligned} \tag{2.5}$$

Where the last step is possible because the impulse response matrix is deterministic. By using the hypothesis in Eq. 2.1,

$$\int_{-\infty}^{\infty}\int_{-\infty}^{\infty}\mathbf{H}(\tau_1)\mathbf{I}\delta(\tau_2-\tau_1)\mathbf{H}^T(\tau_2)\,d\tau_1\,d\tau_2 \tag{2.6}$$

Which proves how the autocorrelation of $\boldsymbol{z}$ is not time-dependent and therefore can be brought out of the integral and limit.

By substituting this into the previous equation:

$$\lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} \underbrace{\operatorname{trace}\left\{E\left[(\mathbf{z}(t)\mathbf{z}(t))^T\right]\right\}}_{\text{NOT TIME DEP.}} dt = \tag{2.7}$$

$$= \lim_{T\to\infty} \frac{1}{2T} \cdot 2T \cdot \operatorname{trace}(R_{zz}(t,t)) = \tag{2.8}$$

$$= \operatorname{trace} E\left[(\mathbf{z}(t)\mathbf{z}(t))^T\right] \tag{2.9}$$

By using the equation of $R_{zz}$, this is equivalent to:

$$= \operatorname{trace}\left\{\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} \mathbf{H}(\tau_1)\mathbf{I}\delta(\tau_2-\tau_1)\mathbf{H}^T(\tau_2)\, d\tau_1\, d\tau_2\right\} = \tag{2.10}$$

$$= \operatorname{trace}\left\{\int_{-\infty}^{\infty} \mathbf{H}(\tau_1)\mathbf{I}\left(\int_{-\infty}^{\infty} \delta(\tau_2-\tau_1)\mathbf{H}^T(\tau_2)\, d\tau_2\right) d\tau_1\right\} = \tag{2.11}$$

$$= \operatorname{trace}\left\{\int_{-\infty}^{\infty} \mathbf{H}(\tau_1)\mathbf{H}^T(\tau_1)\, d\tau_1\right\} \tag{2.12}$$

$$\tag{2.13}$$

By Parseval's theorem, it's possible to move to the frequency domain equivalent:

$$= \operatorname{trace}\left\{\frac{1}{2\pi}\int_{-\infty}^{\infty} \mathbf{F}(j\omega)\mathbf{F}^H(j\omega)\, d\omega\right\} = \tag{2.14}$$

$$= \frac{1}{2\pi}\int_{-\infty}^{\infty} \operatorname{trace}\left[\mathbf{F}(j\omega)\mathbf{F}^H(j\omega)\right] d\omega = \|\mathbf{F}\|_2^2 = \|\mathbf{F}_l(\mathbf{P},\mathbf{K})\|_2^2 \tag{2.15}$$

Where the definition of $\mathcal{H}_2$ norm of a system and the definition of $\boldsymbol{F}$ have been used. This proves that by minimizing the $\mathcal{H}_2$ norm of $\boldsymbol{F}$, we are minimizing the root mean square value of $\boldsymbol{z}$ under unit intensity white noise input, as discussed in [5].

## 2.2: System zero and down squaring

Given the following state-space system:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{u}$$
$$y = \begin{bmatrix} 1 & -1 \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} 1 & 1 \end{bmatrix} \boldsymbol{u}$$

Whose transfer function matrix is:

$$\boldsymbol{G}(s) = \frac{1}{(s-2)(s+2)} \begin{bmatrix} (s-1)(s+2) & (s+1)(s-2) \end{bmatrix} \tag{2.16}$$

From this formulation, it's easy to see that the zero polynomial (as defined previously) is $z(s) = 1$, meaning we don't have any transmission zeros. If we add a pre-compensator of the type $\boldsymbol{k} = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}$, the state-space system becomes:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{k}v \tag{2.17}$$

$$y = \begin{bmatrix} 1 & -1 \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} 1 & 1 \end{bmatrix} \boldsymbol{k}v \tag{2.18}$$

where $\boldsymbol{u} = \boldsymbol{k}v$. The transfer function becomes one-dimensional:

$$G(s) = \frac{(k_1+k_2)s^2 + (k_1-k_2)s - 2(k_1+k_2)}{(s-2)(s+2)} \tag{2.19}$$

The zeros of the system are the roots of Eq. 2.20:

$$(k_1+k_2)s^2 + (k_1-k_2)s - 2(k_1+k_2) = as^2 + bs + c = 0 \tag{2.20}$$

It's simple to prove that the two roots $s_1$ and $s_2$ of the second degree polynomial satisfy the equation:

$$s_1 \cdot s_2 = \frac{c}{a} = \frac{-2(k_1+k_2)}{(k_1+k_2)} = -2 \tag{2.21}$$

This means that the product of the two zeros is negative, and therefore it is impossible to place both the zeros of the system in the same open half of the complex plane. The transmission zeros computed for this system with the procedure explained in section 1.1 for non-square system can be seen in the code `Ex_2_2_2.m`.

## 2.3: Stabilization via Youla-controller

Following the procedure from ACS, Exercise 07, a stabilizing Youla controller was designed by finding a coprime factorization using:

$$\begin{bmatrix} \mathbf{M} & \mathbf{U} \\ \mathbf{N} & \mathbf{V} \end{bmatrix} \overset{s}{=} \left[\begin{array}{c|c} \mathbf{A}+\mathbf{BF} & \begin{bmatrix} \mathbf{B} & -\mathbf{H} \end{bmatrix} \\ \hline \begin{bmatrix} \mathbf{F} \\ \mathbf{C}+\mathbf{DF} \end{bmatrix} & \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{D} & \mathbf{I} \end{bmatrix} \end{array}\right], \tag{2.22}$$

Where $\boldsymbol{F}$ and $\boldsymbol{H}$ were found by using the MATLAB `place` function. The poles were placed in $\begin{bmatrix} -2, -5, -8 \end{bmatrix}$. If needed, they can be moved further to the left of the complex plane to make the response of the system faster:

```
F = -place(A, B, [-2 -5 -8]);
H = -transpose(place(A' , C' , [ -2 -5 -8]));
```

The obtained coprime factors are:

$$M(s) = \frac{(s-2)(s+2)(s+1)}{(s+8)(s+5)(s+2)} \tag{2.23}$$

$$N(s) = \frac{s-1}{(s+8)(s+5)(s+2)} \tag{2.24}$$

$$U(s) = \frac{-6664(s+2)(s+0.9412)}{(s+8)(s+5)(s+2)} \tag{2.25}$$

$$V(s) = \frac{(s-7.704)(s^2+36.7s+614.8)}{(s+8)(s+5)(s+2)} \tag{2.26}$$

The stabilizing controllers are parametrized through the transfer function $\boldsymbol{Q} \in \mathcal{RH}_\infty$:

$$\boldsymbol{K} = -(\boldsymbol{V} + \boldsymbol{Q}\boldsymbol{N})^{-1}(\boldsymbol{U} + \boldsymbol{Q}\boldsymbol{M}) \tag{2.27}$$

By setting $\boldsymbol{Q} = \boldsymbol{0}$, the resulting controller has the following transfer function:

$$K(s) = \frac{6664(s+2)(s+0.9412)}{(s-7.704)(s^2+36.7s+614.8)} \tag{2.28}$$

Since the system is SISO, to check stability it is only necessary to check one transfer function. In particular, we have to check that the poles of $(1+GK)^{-1}$ lie in the closed-LHP. The poles have been computed as the roots of the pole polynomial, and only stable poles were found. The step response of the system is shown in Figure 2.1. The procedure and results described in this section can be found in the code `Ex_2_2_3.m`.

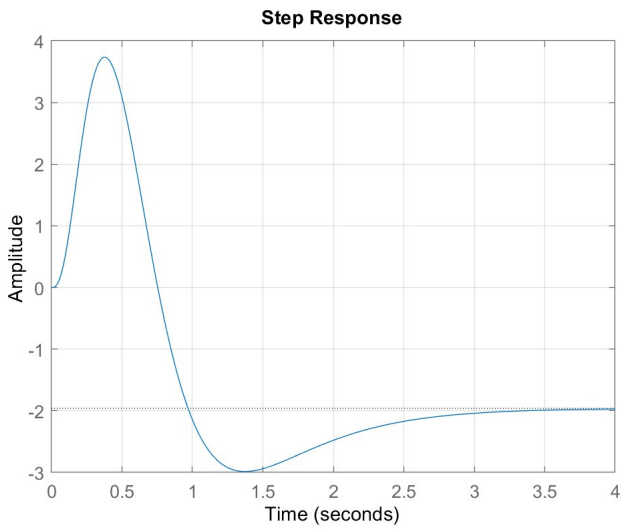


Figure 2.1: System response with Youla-controller.

# Task 3: Stability and LFT

### 3.1: Stability and LFT

Given the following dynamical system:

$$\begin{bmatrix}\dot{x}_1\\ \dot{x}_2\end{bmatrix} = \begin{bmatrix}0 & 1\\ -\frac{k_p}{m_p} & -\frac{c_p}{m_p}\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} + \begin{bmatrix}0\\ \frac{1}{m_p}\end{bmatrix}u$$
$$y = \begin{bmatrix}h_p & 0\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} \tag{3.1}$$

Where $m_p, c_p, , k_p$ and $h_p$ are uncertain parameters described in Table 3.1.

| **Parameter** | **Minimum Value** | **Nominal Value** | **Maximum Value** |
|---|---|---|---|
| $m$ | 5 | 10 | 18 |
| $c$ | 6 | 10 | 14 |
| $k$ | 0.8 | 1 | 1.8 |
| $h$ | 2.0 | 2.5 | 4.5 |

Table 3.1: Parameter Uncertainty Ranges

It's possible to parametrize this uncertainty as a function of four uncertainty channels $\Delta_m, \Delta_c, \Delta_k$ and $\Delta_h$ for asymmetric ranges. For example, we can write $k_p$ as:

$$k_p = \frac{a + b\Delta_k}{c + d\Delta_k} \tag{3.2}$$

where $a, b, c$ and $d$ are determined by imposing:

$$\begin{cases}\Delta_k = -1 & k_p = k_{\text{min}}\\ \Delta_k = 0 & k_p = k_{\text{nom}}\\ \Delta_k = 1 & k_p = k_{\text{max}}\end{cases} \tag{3.3}$$

This is not enough equations to determine all four variables, so $c$ is arbitrarily fixed to 1. The same can be done for all other parameters, including $c_p$ which has a symmetric range. The results are the following:

$$
\begin{aligned}
m_p &= \frac{10 + 3.846\Delta_m}{1 - 0.231\Delta_m} \\
c_p &= c_{\text{nom}}(1 + 0.4\Delta_c) \\
k_p &= \frac{1 - 0.28\Delta_k}{1 - 0.6\Delta_k} \\
h_p &= \frac{2.5 - 0.7\Delta_h}{1 - 0.6\Delta_h}
\end{aligned}
\tag{3.4}
$$

Having obtained the parametrization of the uncertainties, it's possible to re-write them as the LFT between a matrix $\boldsymbol{M}_i$ and the uncertainty $\Delta_i$ (except for $c_p$). For the general case:

$$
\begin{aligned}
\frac{a}{c + d\Delta_i} + \frac{b\Delta_i}{c + d\Delta_i} &= \frac{a}{c}\frac{1 + d\Delta_i/c - d\Delta_i/c}{1 + d\Delta/c} + \frac{b\Delta_i/c}{1 + d\Delta_i/c} \\
&= \frac{a}{c} - \frac{a}{c}\frac{d\Delta_i/c}{1 + d\Delta_i/c} + \frac{b\Delta_i/c}{1 + d\Delta_i/c} \\
&= \frac{a}{c} + \left(b - \frac{ad}{c}\right)\Delta_i\left(1 + \frac{d\Delta_i}{c}\right)^{-1} \cdot \frac{1}{c}
\end{aligned}
\tag{3.5}
$$

The upper LFT is defined as:

$$
\mathcal{F}_u(\boldsymbol{M}_i, \boldsymbol{\Delta}_i) := \boldsymbol{M}_{i,22} + \boldsymbol{M}_{i,21}\boldsymbol{\Delta}_i(\boldsymbol{I} - \boldsymbol{M}_{i,11}\boldsymbol{\Delta}_i)^{-1}\boldsymbol{M}_{i,12} \tag{3.6}
$$

Therefore, in this case, matrix $\boldsymbol{M}_i$ is defined as:

$$
\boldsymbol{M}_i = \begin{bmatrix} -\frac{d}{c} & \frac{1}{c} \\ b - \frac{ad}{c} & \frac{a}{c} \end{bmatrix} \tag{3.7}
$$

This was performed for the uncertainties $m_p, k_p$ and $h_p$. In particular, since $m_p$ always appears at the denominator, the parametrization was performed on $1/m_p$. The results are the following:

$$
\begin{aligned}
\boldsymbol{M}_m &= \begin{bmatrix} -0.3846 & 0.1 \\ -0.6156 & 0.1 \end{bmatrix} \\
\boldsymbol{M}_k &= \begin{bmatrix} 0.6 & 1 \\ 0.32 & 1 \end{bmatrix} \\
\boldsymbol{M}_h &= \begin{bmatrix} 0.6 & 1 \\ 0.8 & 2.5 \end{bmatrix}
\end{aligned}
\tag{3.8}
$$

The new block diagram of this system can be now drawn and is shown in Figure 3.1.

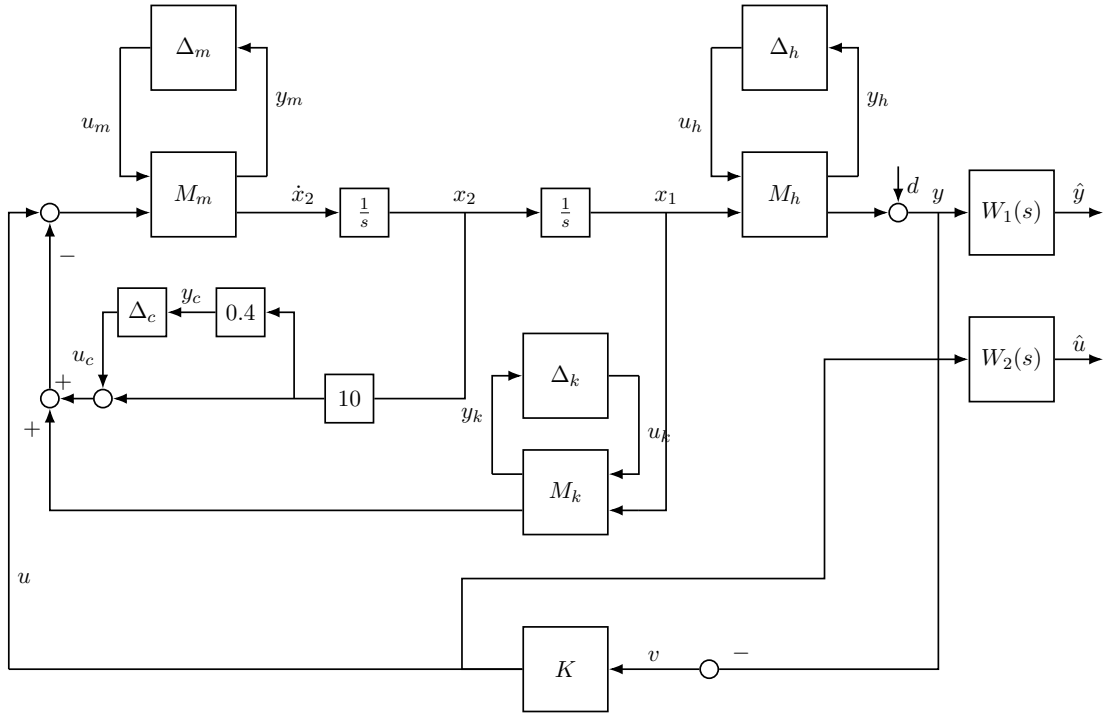


Figure 3.1: Block diagram after uncertainty extraction.

From here it's possible to re-arrange the system to an $\Delta$-$P$-$K$ structure shown in Figure 3.2. $\boldsymbol{P}$ has to be a system such that:

$$\begin{bmatrix} y_m \\ y_c \\ y_k \\ y_h \\ \hat{y} \\ \hat{u} \\ v \end{bmatrix} = \boldsymbol{P} \begin{bmatrix} u_m \\ u_c \\ u_k \\ u_h \\ d \\ u \end{bmatrix} \tag{3.9}$$

Where $\hat{y}$ and $\hat{u}$ are the output $y$ and input $u$ scaled by the weighting functions $W_1(s)$ and $W_2(s)$, respectively. By looking at the block diagram, such system can be derived. Here it's the state-space representation:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{z}_y \\ \dot{z}_u \end{bmatrix} = \boldsymbol{A} \begin{bmatrix} x_1 \\ x_2 \\ z_y \\ z_u \end{bmatrix} + \boldsymbol{B} \begin{bmatrix} u_m \\ u_c \\ u_k \\ u_h \\ d \\ u \end{bmatrix}$$

$$\begin{bmatrix} y_m \\ y_c \\ y_k \\ y_h \\ \hat{y} \\ \hat{u} \\ v \end{bmatrix} = \boldsymbol{C} \begin{bmatrix} x_1 \\ x_2 \\ z_y \\ z_u \end{bmatrix} + \boldsymbol{D} \begin{bmatrix} u_m \\ u_c \\ u_k \\ u_h \\ d \\ u \end{bmatrix}$$

where:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -M_{m,22}M_{k,22} & -M_{m,22}\bar{c} & 0 & 0 \\ B_{w1}M_{h,22} & 0 & A_{w1} & 0 \\ 0 & 0 & 0 & A_{w2} \end{bmatrix}$$

$$\boldsymbol{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ M_{m,21} & -M_{m,22} & -M_{k,21}M_{m,22} & 0 & 0 & M_{m,22} \\ 0 & 0 & 0 & B_{w1}M_{h,21} & B_{w1} & 0 \\ 0 & 0 & 0 & 0 & 0 & B_{w2} \end{bmatrix}$$

$$\boldsymbol{C} = \begin{bmatrix} -M_{m,12}M_{k,22} & -M_{m,12}\bar{c} & 0 & 0 \\ 0 & 0.4\bar{c} & 0 & 0 \\ M_{k,12} & 0 & 0 & 0 \\ M_{h,12} & 0 & 0 & 0 \\ D_{w1}M_{h,22} & 0 & C_{w1} & 0 \\ 0 & 0 & 0 & C_{w2} \\ -M_{h,22} & 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{D} = \begin{bmatrix} M_{m,11} & -M_{m,12} & -M_{m,12}M_{k,21} & 0 & 0 & M_{m,12} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & M_{k,11} & 0 & 0 & 0 \\ 0 & 0 & 0 & M_{h,11} & 0 & 0 \\ 0 & 0 & 0 & D_{w1}M_{h,21} & D_{w1} & 0 \\ 0 & 0 & 0 & 0 & 0 & D_{w2} \\ 0 & 0 & 0 & -M_{h,21} & -1 & 0 \end{bmatrix}$$

The reader may notice how there are two extra states in this system. This is because the two weighting functions are dynamic systems that have as state-space equations:

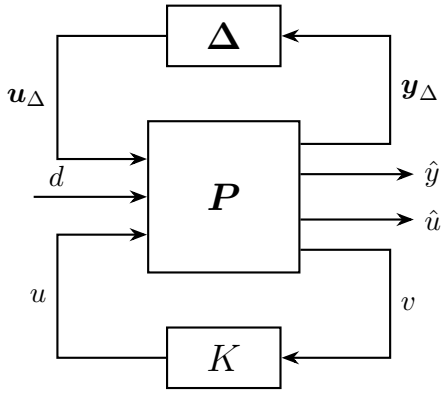


Figure 3.2: $\Delta$-$P$-$K$ Structure.

$$\begin{aligned} \dot{z}_y &= A_{w1} z_y + B_{w1} y \\ \hat{y} &= c_{w1} z_y + C_{w1} y \end{aligned} \tag{3.10}$$

$$\begin{aligned} \dot{z}_u &= A_{w2} z_u + B_{w2} u \\ \hat{u} &= c_{w2} z_u + C_{w2} u \end{aligned} \tag{3.11}$$

While the matrix $\mathbf{\Delta}$ is defined as:

$$\mathbf{\Delta} = \begin{bmatrix} \Delta_m & 0 & 0 & 0 \\ 0 & \Delta_c & 0 & 0 \\ 0 & 0 & \Delta_c & 0 \\ 0 & 0 & 0 & \Delta_h \end{bmatrix} \tag{3.12}$$

A similar result could have been obtained through the *Robust Control Toolbox* in MATLAB. Without writing explicitly the uncertainties, the equations of the system are:

$$\begin{aligned} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{z}_y \\ \dot{z}_u \end{bmatrix} &= \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_p}{m_p} & -\frac{c_p}{m_p} & 0 & 0 \\ B_{w_1} h_p & 0 & A_{w_1} & 0 \\ 0 & 0 & 0 & A_{w_2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ z_y \\ z_u \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{m_p} \\ B_{w_1} & 0 \\ 0 & B_{w_2} \end{bmatrix} \begin{bmatrix} d \\ u \end{bmatrix} \\ \begin{bmatrix} \hat{y} \\ \hat{u} \\ v \end{bmatrix} &= \begin{bmatrix} D_{w_1} h_p & 0 & C_{w_1} & 0 \\ 0 & 0 & 0 & C_{w_2} \\ -h_p & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ z_y \\ z_u \end{bmatrix} + \begin{bmatrix} D_{w_1} & 0 \\ 0 & D_{w_2} \\ -1 & 0 \end{bmatrix} \begin{bmatrix} d \\ u \end{bmatrix} \end{aligned} \tag{3.13}$$

Through the function `ureal`, it's possible to define a uncertain real variable by specifying its range and nominal value.

```
m = ureal('m', 10, 'Range', [5, 18]);
```

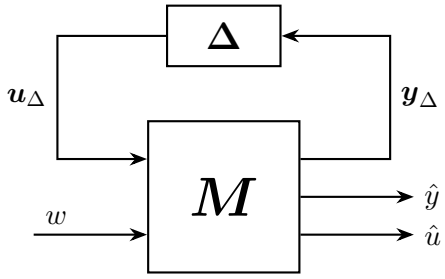


Figure 3.3: $M$-$\Delta$ Structure.

```
c = ureal('c', 10, 'Range', [6, 14]);
k = ureal('k', 1, 'Range', [0.8, 1.8]);
h = ureal('h', 2.5, 'Range', [2, 4.5]);
```

Then, Eq.3.13 was written in MATLAB by creating an uncertain state-space system named `Plant_aut`. Through the command `lftdata`, a $P$-$\Delta$ structure was found:

```
[P_aut, Delta_aut] = lftdata(Plant_aut);
```

Differently from before, where $\boldsymbol{\Delta}$ had four uncertainty channels, with this procedure it has six. This is most likely because MATLAB repeats three times the uncertainty of $m_p$.

A controller was designed to satisfy robust stability and nominal performance. The controller designed is the following:

$$K = \frac{0.3\left(\frac{s}{0.88729}+1\right)\left(\frac{s}{0.1127}+1\right)\left(\frac{s}{48}+1\right)}{s\left(\frac{s}{2}+1\right)^2} \tag{3.14}$$

To check for robust stability, both the plant computed manually and the one computed automatically through the *Robust Control Toolbox* were rearranged to the $\boldsymbol{M}$-$\boldsymbol{\Delta}$ structure shown in Figure 3.3. This was done in MATLAB by using the `lft` function by using the following syntax:

```
M = lft(P, K);
M_aut = lft(P_aut, K);
```

Robust stability is satisfied for the small gain theorem if:

$$\|\boldsymbol{M}_{11}\|_\infty \leq 1 \tag{3.15}$$

Unfortunately, for both types of uncertainty modeling, this requirement is not satisfied. The norm in the two cases is 3.46 and 11.10, respectively. Due to the increased number

of uncertainty channels, the system obtained using the *Robust Control Toolbox* has more conservatism, as it can be seen by the increased norm.

Another degree of conservatism is introduced by Eq. 3.15. This constraint doesn't consider the type of uncertainty the system is subject to. To account for this, another procedure was implemented using the Structured Singular Value. This way, the type of uncertainty can be specified as real and diagonal. Let's consider the following code:

```
Nfr = frd(minreal(M(1:4, 1:4)), logspace(-3, 3, 100));
[bounds, muInfo] = mussv(Nfr, [-1, 1; -1, 1; -1, 1; -1, 1]);
mu = bounds(1);

if all(mu.ResponseData<1)
    disp("The system is robustly stable");
else
    disp("The system may NOT be robustly stable");
end
```

In line 2, we are computing the structured singular value of $\boldsymbol{M}_{11}$, which is the inverse of the smallest structured $\boldsymbol{\Delta}$ that causes instability. Therefore, to have robust stability, this uncertainty has to be larger than the one our system is subject to. This is what the `if` statement is checking for. This procedure was repeated for both types of uncertainty, and in both cases the system is robustly stable.

Nominal performance, instead, is guaranteed if:

$$\|\boldsymbol{M}_{22}\|_\infty \leq 1 \tag{3.16}$$

Another way of looking at this is to check the two outputs separately. The transfer function of the original system is:

$$G_p = \frac{h_p}{m_p s^2 + c_p s + k_p} \tag{3.17}$$

By evaluating this transfer function at the nominal values, we can re-write the nominal performance requirements as:

$$\|W_1(1 + G_p K)^{-1}\|_\infty = \|W_1 S\|_\infty \leq 1 \tag{3.18}$$

for the output, and:

$$\|W_2 K(1 + G_p K)^{-1}\|_\infty = \|W_2 K S\|_\infty \leq 1 \tag{3.19}$$

for the control effort. As shown in Figure 3.4, these requirements are not entirely satisfied: the norms of Eq. 3.16 for the system computed manually and automatically are 3.64919 and 3.64922, respectively. In this case the two norms are very close, which is expected because the nominal plants should be the same, as the difference is only in the way they handle uncertainty.

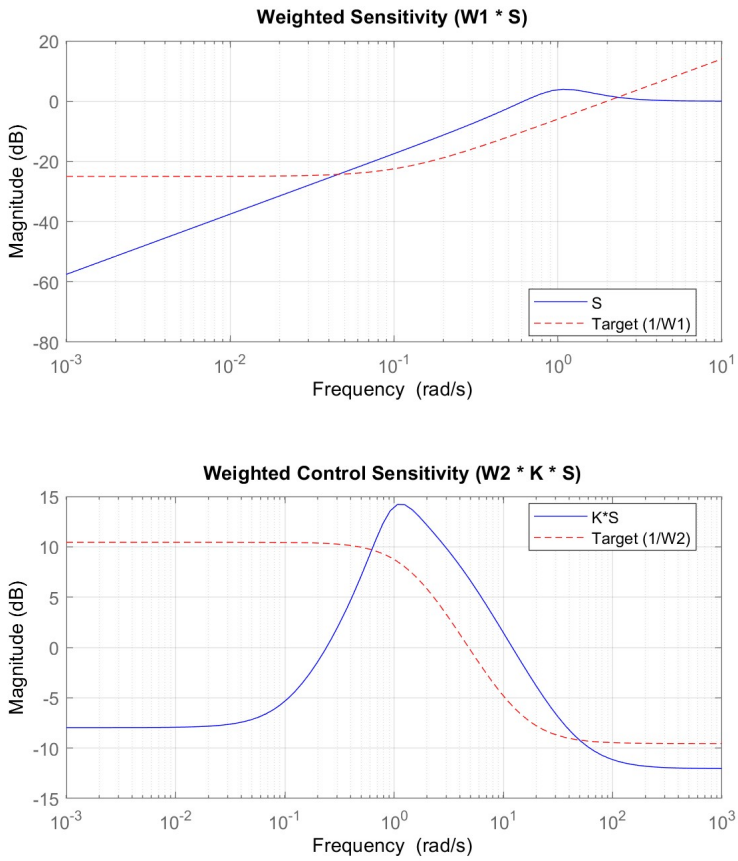


Figure 3.4: Bode plots of the sensitivity and control sensitivity, compared to the shape they should have to satisfy nominal performance.

Consider now a time-delay in the transfer function $G_p$, as a first-order Padé approximation:

$$G_p = \frac{h_p}{m_p s^2 + c_p s + k_p} e^{-\tau_p s} \approx \frac{h_p}{m_p s^2 + c_p s + k_p} \frac{1 - \frac{\tau_p}{2} s}{1 + \frac{\tau_p}{2} s} \tag{3.20}$$

Where $\tau_p$ is an uncertain parameter with nominal value $\bar{\tau}$ and range $[0.07, 0.13]$.
The transfer function matrix has one input $d$ and two outputs ($\hat{y}$ and $\hat{u}$):

$$\boldsymbol{G}_{\text{delay}} = \begin{bmatrix} W_1(1 + G_p K)^{-1} \\ W_2 K(1 + G_p K)^{-1} \end{bmatrix} \tag{3.21}$$

As shown in the code , this transfer function matrix can be converted into an uncertain state-space representation and through the `lftdata` command, it's possible to recover the $\boldsymbol{M}$-$\boldsymbol{\Delta}$ structure:

```
[M_delay, Delta_delay] = lftdata(G_delay);
n_delay = length(Delta_delay.NominalValue);
M_11 = M_delay(1:n_delay, 1:n_delay);
```

The robust stability criterion shown in 3.15, is not satisfied, so it's impossible to say whether this system is robustly stable with the small gain theorem. By using the less conservative procedure with the structured singular values shown in the previous snippet of code, the system is proved to be robustly stable even with the delay. If the reader is interested in the complete code, it can be found in the file `Ex_2_3_1.m`.

### 3.2: Sensitivity integrals

The following proof was found on chapter 6.2 of [2]. The starting point is Lemma 3, which states that for every point $s_0 = \sigma_0 + j\omega$ with $\sigma_0 > 0$,

$$\log|S_{\text{mp}}(s_0)| = \frac{1}{\pi}\int_{-\infty}^{\infty} \log|S(j\omega)| \frac{\sigma_0}{\sigma_0^2 + (\omega - \omega_0)^2} d\omega. \tag{3.22}$$

Where $S = S_{\text{ap}} S_{\text{mp}}$ where $S_{\text{ap}}$ is an all-pass function and $S_{\text{mp}}$ is a minimum phase function. By taking $\omega_0 = 0$:

$$\log|S_{\text{mp}}(\sigma_0)| = \frac{1}{\pi}\int_{-\infty}^{\infty} \log|S(j\omega)| \frac{\sigma_0}{\sigma_0^2 + \omega^2} d\omega \tag{3.23}$$

Or, equivalently:

$$\int_0^{\infty} \log|S(j\omega)| \frac{\sigma_0^2}{\sigma_0^2 + \omega^2} d\omega = \frac{\pi}{2}\sigma_0 \log|S_{\text{mp}}(\sigma_0)| \tag{3.24}$$

Since $\lim_{\sigma_0\to\infty} \frac{\sigma_0^2}{\sigma_0^2+\omega^2} = 1$ and $\lim_{\omega\to\infty} \log|S(j\omega)| = 0$, the left hand converges to:

$$\int_0^{\infty} \log|S(j\omega)| d\omega \tag{3.25}$$

So, it remains to show that:

$$\lim_{\sigma\to\infty} \frac{\sigma}{2} \log|S_{\text{mp}}(\sigma)| = (\log e)\left(\sum \text{Re } p_i\right) \tag{3.26}$$

It's possible to write the non-minimum phase part as:

$$S_{\text{ap}}(s) = \prod_i \frac{s - p_i}{s + \bar{p}_i} \tag{3.27}$$

Since $L$ has a relative degree of at least 2:

$$L(\sigma) \approx \frac{c}{\sigma^k} \quad \text{as } \sigma \to \infty \tag{3.28}$$

for some constant $c$ and some integer $k \geq 2$. Thus, as $\sigma \to \infty$:

$$\sigma \ln S(\sigma) = -\sigma \ln 1 + L(\sigma) \approx -\sigma \ln \left(1 + \frac{c}{\sigma^k}\right) \tag{3.29}$$

Now use the Maclaurin's series:

$$1 + \mathrm{x} = x - \frac{x^2}{2} + \cdots \tag{3.30}$$

to get:

$$\sigma \ln S(\sigma) \approx -\sigma \left(\frac{c}{\sigma^k} - \cdots\right) \tag{3.31}$$

Which proves that:

$$\lim_{\sigma \to \infty} \sigma \ln S(\sigma) = 0 \tag{3.32}$$

Since $S_{\mathrm{mp}} = S/S_{\mathrm{ap}}$, to prove Eq. 3.26, it remains to show that:

$$\lim_{\sigma \to \infty} \frac{\pi}{2} \sigma \ln \left| \left[ S_{\mathrm{ap}}(\sigma)^{-1} \right] \right| = \sum \mathrm{Re}\ p_i \tag{3.33}$$

Where:

$$\ln \left( S_{\mathrm{ap}}(\sigma)^{-1} \right) = \ln \prod_i \frac{\sigma + \bar{p}_i}{\sigma - p_i} = \sum_i \ln \frac{\sigma + \bar{p}_i}{\sigma - p_i} \tag{3.34}$$

Let $p_i = x + jy$ and use Eq. 3.30 as follows:

$$\begin{aligned}
\frac{\sigma}{2} \ln \left| \frac{\sigma + \bar{p}_i}{\sigma - p_i} \right| &= \frac{\sigma}{2} \ln \left| \frac{1 + \bar{p}_i \sigma^{-1}}{1 - p_i \sigma^{-1}} \right| && (3.35)\\
&= \frac{\sigma}{4} \ln \frac{(1 + x\sigma^{-1})^2 + (y\sigma^{-1})^2}{(1 - x\sigma^{-1})^2 + (y\sigma^{-1})^2} && (3.36)\\
&= \frac{\sigma}{4} \left\{ \ln[(1 + x\sigma^{-1})^2 + (y\sigma^{-1})^2] - \ln[(1 - x\sigma^{-1})^2 + (y\sigma^{-1})^2] \right\} && (3.37)\\
&= \frac{\sigma}{4} \left\{ 2\frac{x}{\sigma} + 2\frac{x}{\sigma} + \cdots \right\} && (3.38)\\
&= x + \cdots && (3.39)\\
&= \mathrm{Re} p_i + \cdots . && (3.40)
\end{aligned}$$

Letting $\sigma \to \infty$ gives:

$$\lim_{\sigma \to \infty} \left| \frac{\sigma + \bar{p}_i}{\sigma - p_i} \right| = \mathrm{Re}\ p_i \tag{3.41}$$

Which proves Eq. 3.34. By putting every step together, it's proved that:

$$\int_0^\infty \log |S(j\omega)| d\omega = \pi (\log e) \left( \sum_i^{N_p} \mathrm{Re}\ p_i \right) \tag{3.42}$$

Or, by the property of logarithms:

$$\int_0^\infty \ln|S(j\omega)|d\omega = \pi\left(\sum_i^{N_p} \text{Re } p_i\right) \tag{3.43}$$

Which is the desired result (both for the base 10 and natural logarithms).
Consider now the following statement from Lemma 2 of Chapter 6.2 of [2]:

> For each function $G$ consisting of stable and proper real rational transfer functions, there exist an all-pass function $G_{ap}$ and a minimum-phase function $G_{mp}$ such that $G = G_{ap}G_{mp}$. The factors are unique up to sign.

There, the following proof is explained.
Let $G_{ap}$ be the product of all factors of the form:

$$\frac{s - s_0}{s + \bar{s_0}} \tag{3.44}$$

where $s_0$ ranges over all zeros of $G$ in $\text{Re}(s) > 0$, counting multiplicities. This is an all-pass filter because its magnitude is 1 anywhere on the imaginary axis. Then define:

$$G_{mp} = \frac{G}{G_{ap}} \tag{3.45}$$

Which completes the proof of existence by construction.

Suppose $G$ has two factorizations, such that:

$$G = G_{ap1}G_{mp1} = G_{ap2}G_{mp2} \tag{3.46}$$

where $G_{ap1}$ and $G_{ap2}$ are all-pass functions, and $G_{mp1}$ and $G_{mp2}$ are minimum-phase functions.
A function $H$ is defined as:

$$H = \frac{G_{ap1}}{G_{ap2}} = \frac{G_{mp2}}{G_{mp1}} \tag{3.47}$$

Since $H$ is the ratio of two all-pass functions, it satisfies $|H(j\omega)| = 1 \quad \forall\omega$ and is a rational all-pass function itself. Rational all-pass function have their zeros located at the mirror images of their poles across the imaginary axis, as defined for the function $G_{ap}$ in the previous part.
Moreover, since $H$ is the ratio of two minimum-phase functions, it's also a minimum-phase function. By definition, a minimum-phase function cannot have any poles or zeros in the open right half-plane. The previous requirement, though imposed symmetry, so it's also impossible for it to have poles in the left half-plane either. It's also impossible to have poles/zeros in the origin without breaking the norm constraint.
Finally, $H$ is a rational function with no poles, no zeros, and with unit norm, which means it has to be: $H(s) = \pm 1$, which proves the factors are unique up to the sign.

# Task 4: Coprime factorization and Youla Parametrization

### 4.1: HIMAT example system analysis

To compute the Smith and Smith-McMillan forms of the reduced system, the following steps were performed:

1. Computing the common denominator: This is the characteristic polynomial of matrix $\boldsymbol{A}_{\text{red}}$:
$$d(s) = \det(s\boldsymbol{I} - \boldsymbol{A}_{\text{red}}) \tag{4.1}$$
Whose roots are the poles of the system.

2. Conversion to polynomial matrix: The MATLAB function `smithForm` requires a polynomial input, but the system's transfer function is rational. Therefore, the transfer function $\boldsymbol{G}(s)$ is converted into the polynomial matrix $\boldsymbol{N}(s)$ by clearing the denominators:
$$\boldsymbol{N}(s) = \boldsymbol{G}(s) \cdot d(s) \tag{4.2}$$

3. Smith Normal Form: The Smith Normal Form of the $\boldsymbol{N}(s)$ is computed through the function `smithForm`, which returns:
$$\boldsymbol{S}(s) = \boldsymbol{U}(s)\boldsymbol{N}(s)\boldsymbol{V}(s) = \text{diag}(\gamma_1'(s), \ldots, \gamma_r'(s)) \tag{4.3}$$
where $\gamma_i'(s)$ are the invariant polynomials such that $\gamma_i'(s)$ divides $\gamma_{i+1}'(s)$.

4. Smith-McMillan Form: The Smith-McMillan form $\boldsymbol{M}(s)$ is obtained by dividing the Smith form back by the common denominator:
$$\boldsymbol{M}(s) = \frac{\boldsymbol{S}(s)}{d(s)} = \text{diag}\left(\frac{\gamma_1(s)}{\beta_1(s)}, \ldots, \frac{\gamma_r(s)}{\beta_r(s)}\right) \tag{4.4}$$
where $\gamma_i(s)$ and $\beta_i(s)$ are coprime polynomials. In particular:
   - The roots of $\prod \gamma_i(s)$ are the transmission zeros.
   - The roots of $\prod \beta_i(s)$ are the system poles.

By looking at the zeros and poles computed this way, it's clear that the pole-zeros cancellations didn't happen due to numerical error. The poles and zeros found through the roots of the numerator and denominator of the Smith-McMillan form are shown in Table 4.1, together with the actual poles and zeros of the reduced system, computed using

`tzero` and `pole`. The poles are repeated twice, and that is because they should have canceled with "fake" zeros which show up only because of small numerical differences that prevent cancellations. Notice also how all three systems are unstable, as they have poles in the open right half-plane.

| Metric | Original System | Reduced System | Smith-McMillan Form |
|---|---|---|---|
| **Zeros** | -0.0210 | -0.0272<br>58.2846 | -3.8830, -0.4742<br>-0.0272, 0.5937<br>0.6951, 58.2846 |
| **Poles** | -0.2578<br>$0.6898 \pm 0.2488i$<br>-5.6757<br>-30.0000 (x2) | -3.8830<br>-0.4742<br>0.5937<br>0.6951 | -3.8830 (x2)<br>-0.4742 (x2)<br>0.5937 (x2)<br>0.6951 (x2) |

Table 4.1: Comparison of Poles/Zeros: Original, Reduced, and Smith-McMillan Results

The Singular Value plots of the transfer function of the original system and the reduced system are shown in Figure 4.1. At low frequency, they are very close to each other, showing that the approximation is good. The difference is at higher frequency, where they are more separated. This is expected from a lower-order approximation, since it's important that the approximation is valid only within a certain bandwidth.

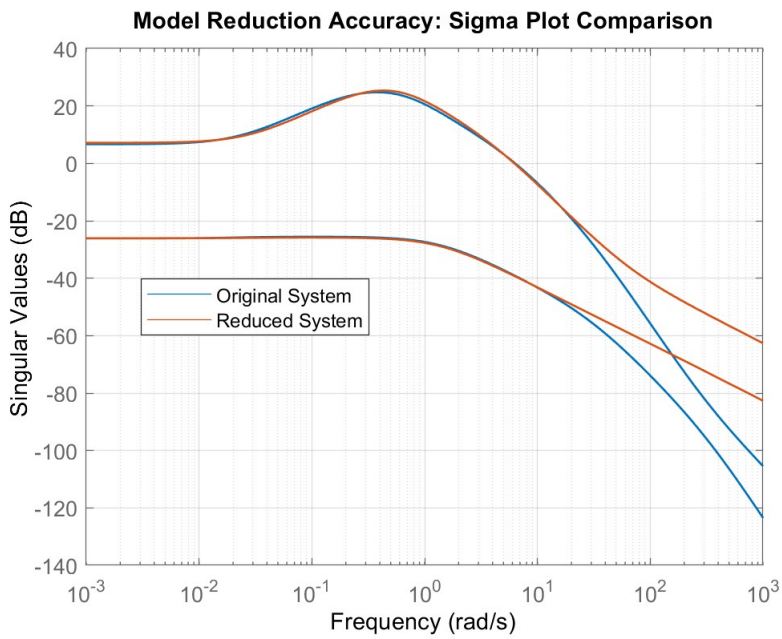


Figure 4.1: Accuracy of the reduced model.

## 4.2: Coprime factorization

A procedure to find a doubly-coprime factorization of a system $\boldsymbol{G}$ is available in [1]. This factorization is formed by four matrices $\boldsymbol{N}, \boldsymbol{M}, \tilde{\boldsymbol{N}}$ and $\tilde{\boldsymbol{M}}$ that satisfy the Eq. 4.5 and Eq. 4.6.

$$\boldsymbol{G} = \boldsymbol{N}\boldsymbol{M}^{-1} = \tilde{\boldsymbol{M}}^{-1}\tilde{\boldsymbol{N}} \tag{4.5}$$

$$\begin{bmatrix} \tilde{\boldsymbol{X}} & -\tilde{\boldsymbol{Y}} \\ -\tilde{\boldsymbol{N}} & \tilde{\boldsymbol{M}} \end{bmatrix} \begin{bmatrix} \boldsymbol{M} & \boldsymbol{Y} \\ \boldsymbol{N} & \boldsymbol{X} \end{bmatrix} = \boldsymbol{I} \tag{4.6}$$

The procedure to find these matrices is the following.

After computing the transfer function of the reduced system through its state-space representation, it's necessary to define a real matrix $\boldsymbol{F}$ such that $\boldsymbol{A}_F := \boldsymbol{A} + \boldsymbol{B}\boldsymbol{F}$ is stable. In order to do this, the MATLAB function `place` was used to place the poles of such system at $\begin{bmatrix} -2, & -5, & -8 \end{bmatrix}$.

Similarly, it's also necessary to define a matrix $\boldsymbol{H}$ such that $\boldsymbol{A}_H = \boldsymbol{A} + \boldsymbol{H}\boldsymbol{C}$ is stable. In the same way as before, the poles were placed at $\begin{bmatrix} -2, & -5, & -8 \end{bmatrix}$. After defining the function $\boldsymbol{B}_H = \boldsymbol{B} + \boldsymbol{H}\boldsymbol{D}$, it's possible to find the matrices that define the matrices that compose the doubly-coprime factorization:

$$\mathbf{M}(s) := \left[\begin{array}{c|c} \mathbf{A}_F & \mathbf{B} \\ \hline \mathbf{F} & \mathbf{I} \end{array}\right] \qquad \mathbf{N}(s) := \left[\begin{array}{c|c} \mathbf{A}_F & \mathbf{B} \\ \hline \mathbf{C}_F & \mathbf{D} \end{array}\right]$$

$$\tilde{\mathbf{M}}(s) := \left[\begin{array}{c|c} \mathbf{A}_H & \mathbf{H} \\ \hline \mathbf{C} & \mathbf{I} \end{array}\right] \qquad \tilde{\mathbf{N}}(s) := \left[\begin{array}{c|c} \mathbf{A}_H & \mathbf{B}_H \\ \hline \mathbf{C} & \mathbf{D} \end{array}\right]$$

$$\mathbf{X}(s) := \left[\begin{array}{c|c} \mathbf{A}_F & -\mathbf{H} \\ \hline \mathbf{C}_F & \mathbf{I} \end{array}\right] \qquad \mathbf{Y}(s) := \left[\begin{array}{c|c} \mathbf{A}_F & -\mathbf{H} \\ \hline \mathbf{F} & \mathbf{0} \end{array}\right]$$

$$\tilde{\mathbf{X}}(s) := \left[\begin{array}{c|c} \mathbf{A}_H & -\mathbf{B}_H \\ \hline \mathbf{F} & \mathbf{I} \end{array}\right] \qquad \tilde{\mathbf{Y}}(s) := \left[\begin{array}{c|c} \mathbf{A}_H & -\mathbf{H} \\ \hline \mathbf{F} & \mathbf{0} \end{array}\right]$$

To check whether this parametrization was correct, it was proved that these matrices respect Bezout's identity shown in Eq. 4.6:

$$\left\| \begin{bmatrix} \tilde{\boldsymbol{X}} & -\tilde{\boldsymbol{Y}} \\ -\tilde{\boldsymbol{N}} & \tilde{\boldsymbol{M}} \end{bmatrix} \begin{bmatrix} \boldsymbol{M} & \boldsymbol{Y} \\ \boldsymbol{N} & \boldsymbol{X} \end{bmatrix} - \boldsymbol{I} \right\|_\infty = 1.2073 \cdot 10^{-7} \tag{4.7}$$

Which is an acceptable numerical error.

## 4.3: Youla parametrization

Once the doubly-coprime factorization of $\boldsymbol{G}$ is available, the set of all (proper-real-rational) controllers $\boldsymbol{K}$ stabilizing $\boldsymbol{G}$ is parametrized by the formulas:

$$\boldsymbol{K} = (\boldsymbol{Y} - \boldsymbol{M}\boldsymbol{Q})(\boldsymbol{X} - \boldsymbol{N}\boldsymbol{Q})^{-1} \tag{4.8}$$
$$= (\tilde{\boldsymbol{X}} - \boldsymbol{Q}\tilde{\boldsymbol{N}})^{-1}(\tilde{\boldsymbol{Y}} - \boldsymbol{Q}\tilde{\boldsymbol{M}}) \tag{4.9}$$

$$\boldsymbol{Q} \in \mathcal{RH}_\infty \tag{4.10}$$

A stabilizing controller can be found, for example, by setting $\boldsymbol{Q} = \boldsymbol{0}$. Using the function `sigma`, it's possible to plot the singular values as function of the frequency. As in our case the system has two inputs and two outputs, we have two singular values both for the sensitivity transfer function and for the complementary sensitivity. The singular value plots are shown in Figure 4.2 and Figure 4.3.

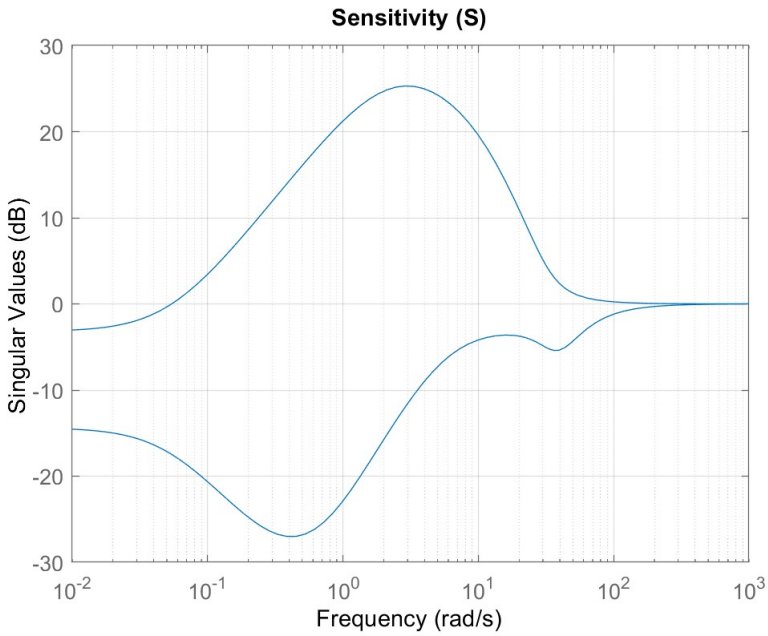


Figure 4.2: Singular Value plot of the Sensitivity transfer function matrix $\boldsymbol{S}(s)$.

The step response of the stabilized system, instead, can be seen in Figure 4.4. If necessary, it's possible to speed up the response of the system by moving the poles of $\boldsymbol{A}_F$ and $\boldsymbol{A}_H$ further to the left in the complex plane, at the expense of having a larger overshoot. All the code referenced in this chapter can be found in the file `Ex_2_4.m`.

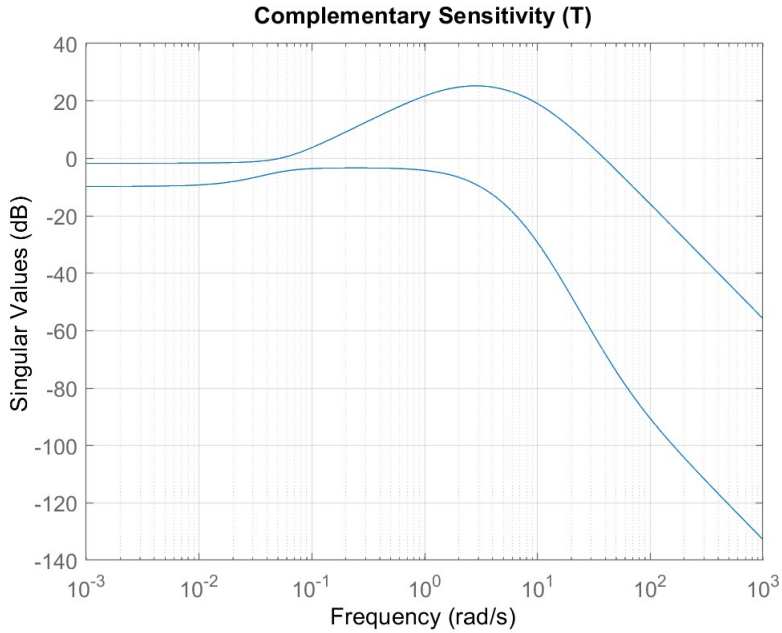


Figure 4.3: Singular Value plot of the Complementary Sensitivity transfer function matrix $\boldsymbol{T}(s)$.

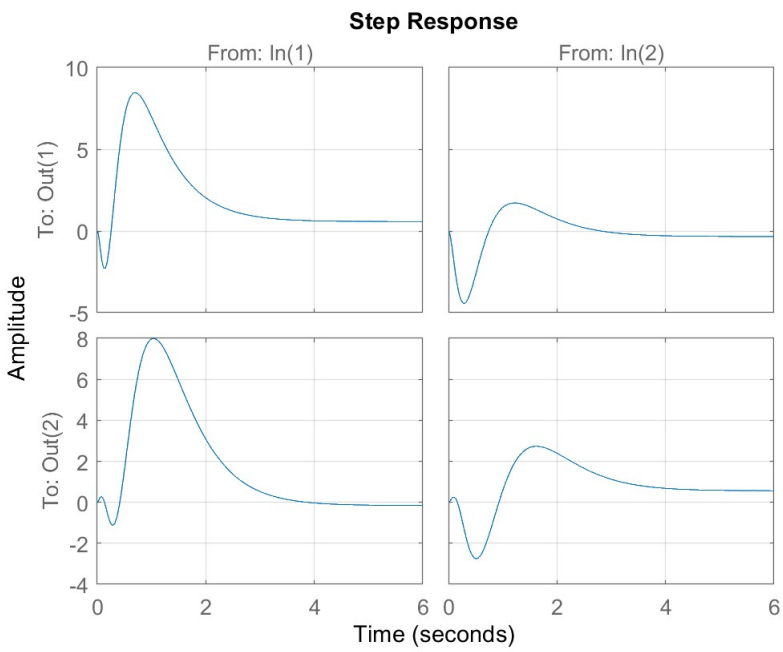


Figure 4.4: Response of the stabilized system.

# Bibliography

[1] B. A. Francis. *A course in H [infinity] control theory.* Springer-Verlag, 1987.

[2] A. T. John Doyle, Bruce Francis. *Feedback Control Theory.* Macmillan Publishing, 1990.

[3] B. Misgeld. Lecture 5: Analysis of multiple-input multiple-output systems II. Lecture script for Advanced Control System, RWTH Aachen University, 2022.

[4] B. Misgeld. Lecture 6: Analysis of multiple-input multiple-output systems III. Lecture script for Advanced Control System, RWTH Aachen University, 2022.

[5] S. Skogestad and I. Postlethwaite. *Multivariable Feedback Control: Analysis and Design.* John Wiley & Sons, 2005.